

Digital Signal Processing Laboratory

Name: Gaukhar

HSM ID: 320941

Date:

Experiment 1: Function Generator and Oscilloscope

Experiment 2: Sine and Dual Tone Signal

Sine signal

```
In [6]: import numpy as np
```

Frequency FFT Frequency

10 kHz

```
In [4]: fs = 8000  
time = np.arange(33.0)/fs
```

```
In [5]: time
```

```
Out[5]: array([0.          , 0.000125, 0.00025 , 0.000375, 0.0005   , 0.000625,  
              0.00075 , 0.000875, 0.001   , 0.001125, 0.00125 , 0.001375,  
              0.0015   , 0.001625, 0.00175 , 0.001875, 0.002   , 0.002125,  
              0.00225 , 0.002375, 0.0025   , 0.002625, 0.00275 , 0.002875,  
              0.003   , 0.003125, 0.00325 , 0.003375, 0.0035   , 0.003625,  
              0.00375 , 0.003875, 0.004   ])
```

```
In [ ]:
```

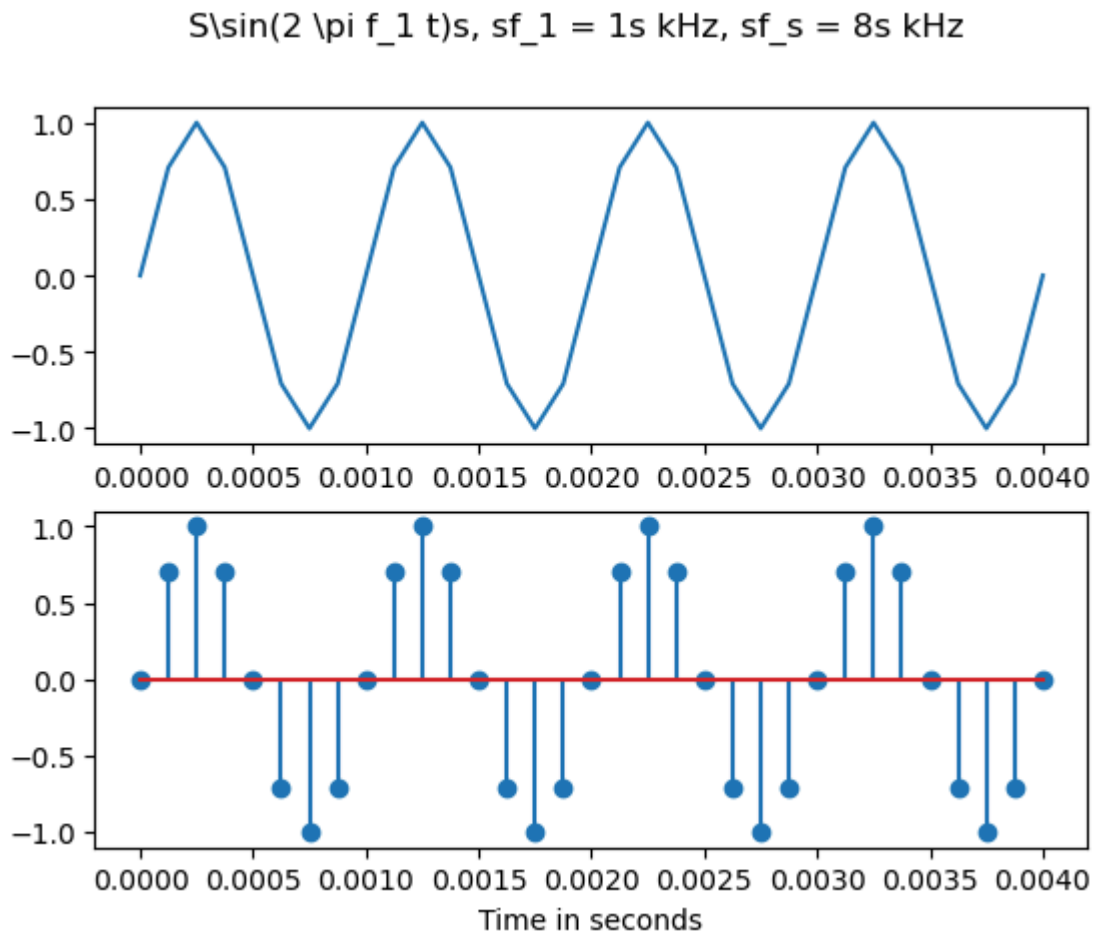
```
In [13]: import numpy as np  
import matplotlib.pyplot as plt  
from IPython.display import Audio
```

```
In [5]: fs = 8000  
f1 = 1000  
time = np.arange(16000.0)/fs  
sine1 = np.sin(2*np.pi*f1*time)
```

```
In [7]: len(sine1)
```

Out[7]: 16000

```
In [19]: plt.figure()
plt.subplot(2,1,1)
plt.plot(time[0:33], sinel[0:33])
plt.subplot(2,1,2)
plt.stem(time[0:33], sinel[0:33])
plt.xlabel('time in seconds')
plt.suptitle('S\sin(2 \pi f_1 t)s, sf_1 = 1s kHz, sf_s = 8s kHz')
plt.show()
```



In [14]: Audio(sinel, rate = fs)

Out[14]:

▶ 0:00 / 0:02 ——— 🔊 ⋮

Dual Tone Signal

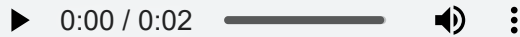
```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Audio
```

```
In [10]: fs = 8000
f1 = 1000
f2 = 1700
time = np.arange(16000.0)/fs
sine2 = np.sin(2*np.pi*f1*time) + 0.5*np.sin(2*np.pi*f2*time)
```

```
In [11]: noise = np.random.normal (0, 0.1, 16000)
        sine3 = sine2 + noise
```

```
In [12]: Audio(sine3, rate = fs)
```

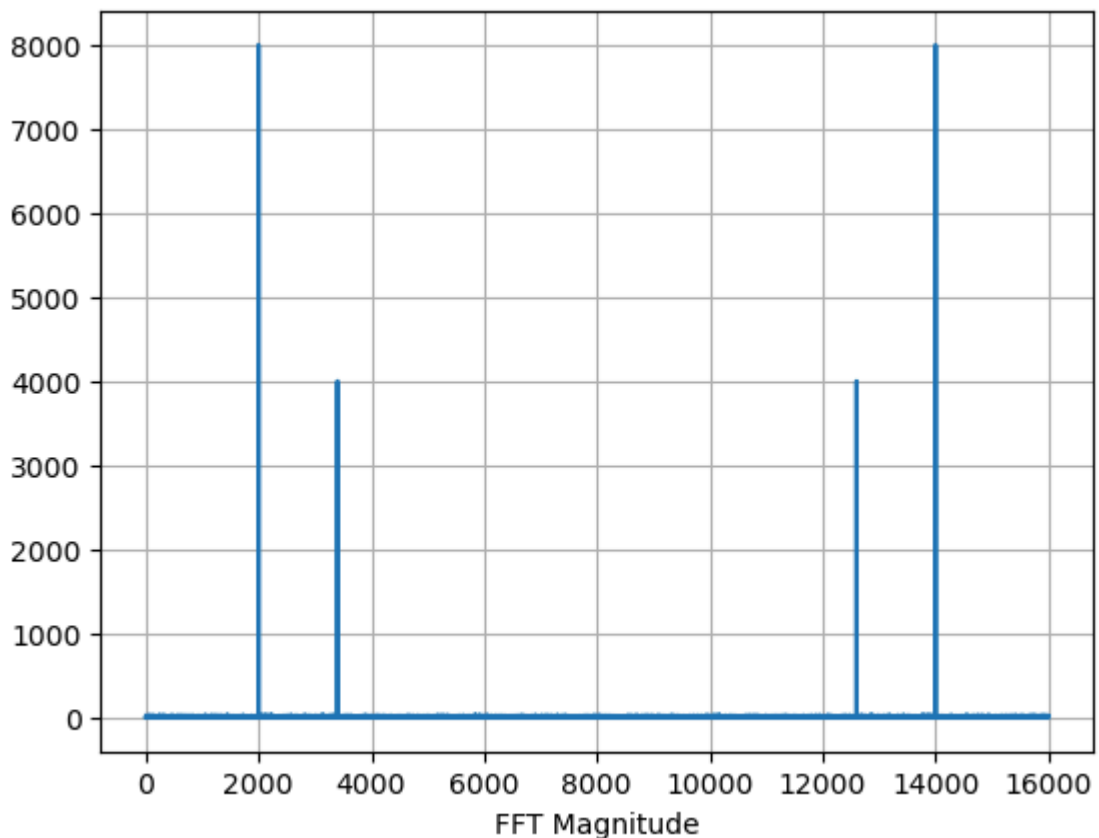
Out[12]:



Spectrum of the dual tone signal

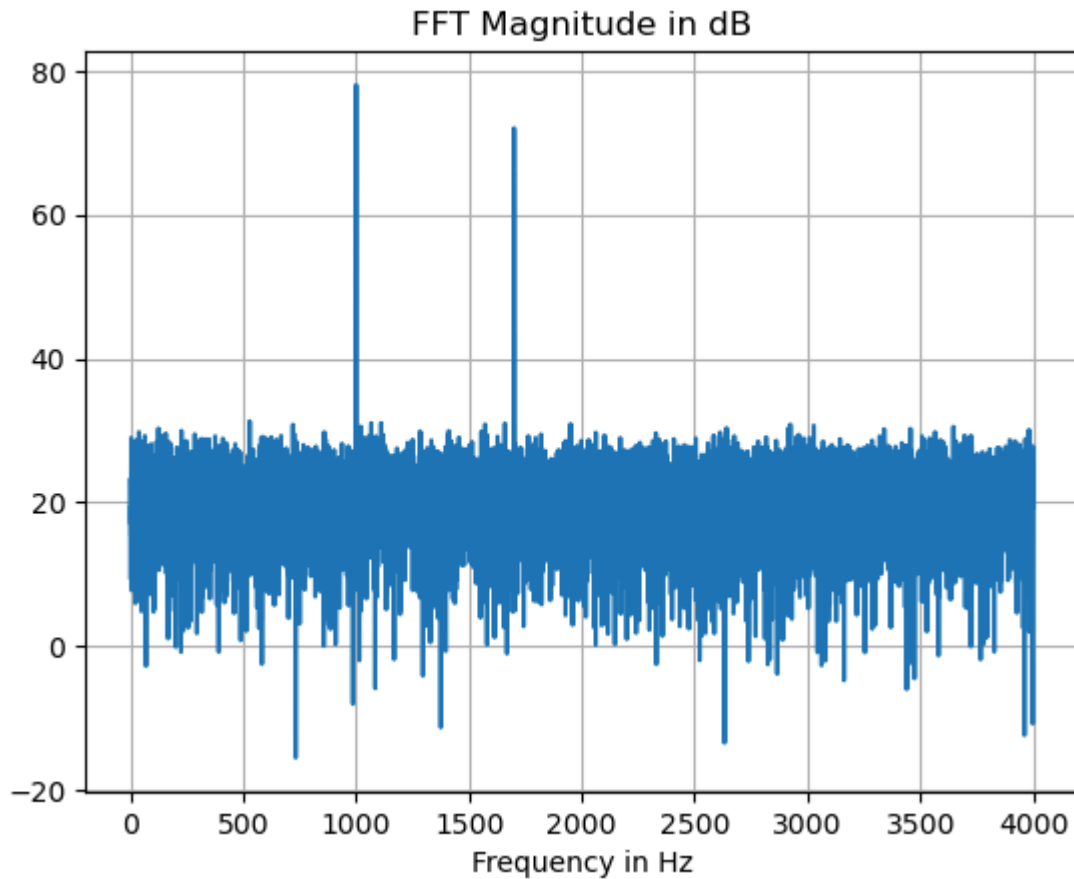
```
In [13]: spec = abs (np.fft.fft(sine3))
```

```
In [14]: plt.plot(spec)
        plt.grid()
        plt.xlabel('FFT Magnitude')
        plt.show()
```



```
In [19]: def plot_spectrum(x, fs):
        # plot the one-sided spectrum (magnitude of DFT) of signal x, scaled in Deci
        # fs: sampling rate
        nsamples = len (x)
        spec = abs (np.fft.fft (x))
        freq_axis = np.arange(float(nsamples))*fs/nsamples
        plt.plot(freq_axis[0:round(nsamples/2)], 20*np.log10(spec[0:round(nsamples/2)]))
        plt.grid()
        plt.xlabel('Frequency in Hz')
        plt.title ('FFT Magnitude in dB')
```

```
In [20]: plot_spectrum(sine3, fs)
```



FIR Filter Design

```
In [ ]: import scipy.signal as scs
```

```
In [26]: n_coeffs = 33
f0_lp = 0.125
b_lp = scs.firwin(n_coeffs, f0_lp)

f0_hp = 0.175
b_hp = scs.firwin(n_coeffs, f0_hp, pass_zero=False)

f_edges = [0.125, 0.225]
b_bp = scs.firwin(n_coeffs, f_edges, pass_zero=False)

w_lp, H_lp = scs.freqz(b_lp)
w_hp, H_hp = scs.freqz(b_hp)
w_bp, H_bp = scs.freqz(b_bp)

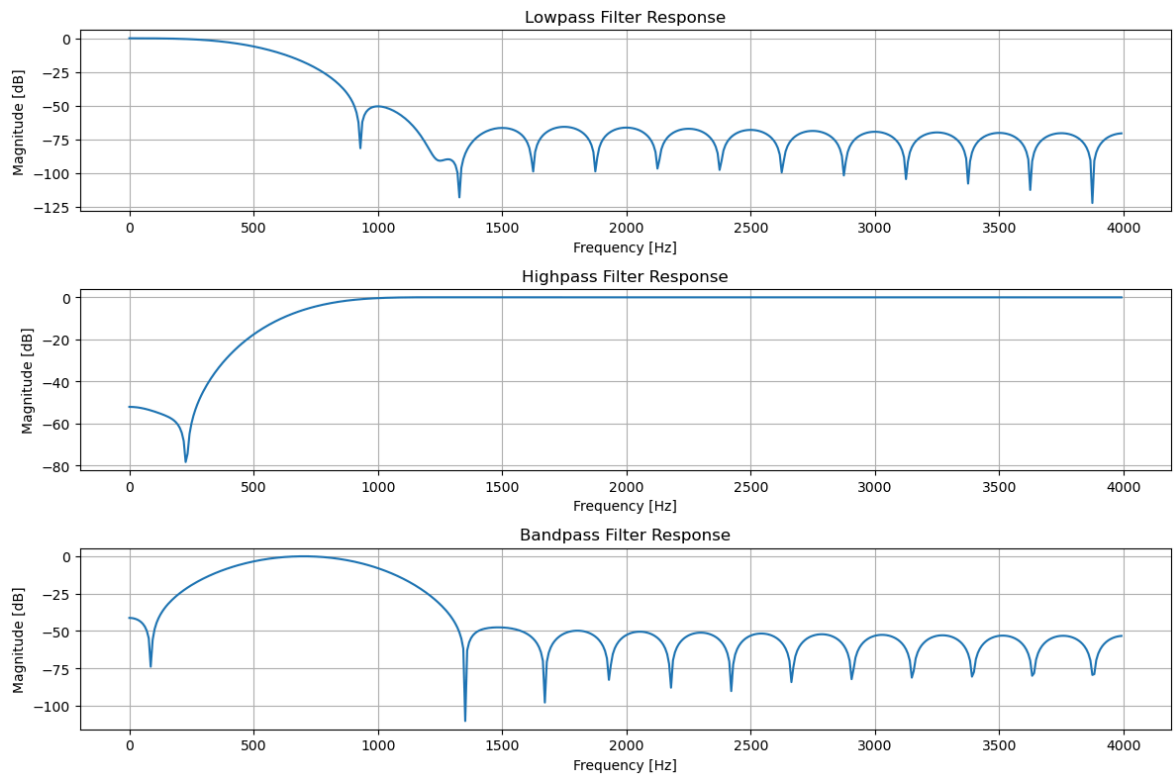
freq_hz = w_lp / np.pi * (fs/2)

plt.figure(figsize=(12, 8))
plt.subplot(3, 1, 1)
plt.plot(freq_hz, 20 * np.log10(np.abs(H_lp)))
plt.title('Lowpass Filter Response')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid()

plt.subplot(3, 1, 2)
plt.plot(freq_hz, 20 * np.log10(np.abs(H_hp)))
plt.title('Highpass Filter Response')
```

```
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid()

plt.subplot(3, 1, 3)
plt.plot(freq_hz, 20 * np.log10(np.abs(H_bp)))
plt.title('Bandpass Filter Response')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid()
plt.tight_layout()
plt.show()
```



```
In [35]: import numpy as np
import scipy.signal as scs
from numpy.fft import fft
import matplotlib.pyplot as plt
from IPython.display import Audio

fs = 8000
f1 = 1000
f2 = 1700
N = 16000

time_dual = np.arange(N) / fs
signal_dual = (np.sin(2 * np.pi * f1 * time_dual) +
               0.5 * np.sin(2 * np.pi * f2 * time_dual) +
               np.random.normal(0, 0.1, N))

n_coeffs = 33
b_lp = scs.firwin(n_coeffs, 0.25)
b_hp = scs.firwin(n_coeffs, 0.35, pass_zero=False)
b_bp = scs.firwin(n_coeffs, [0.25, 0.45], pass_zero=False)
```

```
filtered_lp = scs.lfilter(b_lp, 1, signal_dual)
filtered_hp = scs.lfilter(b_hp, 1, signal_dual)
filtered_bp = scs.lfilter(b_bp, 1, signal_dual)

spec_lp = np.abs(fft(filtered_lp))
spec_hp = np.abs(fft(filtered_hp))
spec_bp = np.abs(fft(filtered_bp))

freq_axis = np.arange(N) * fs / N

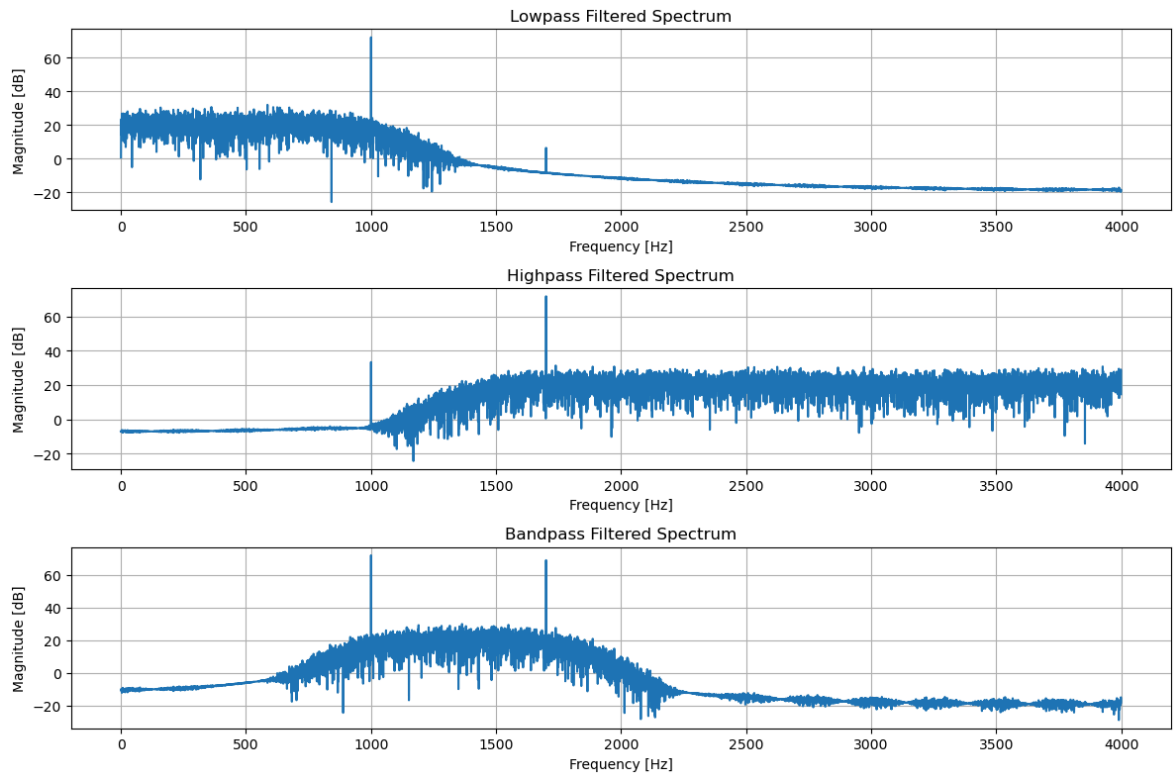
plt.figure(figsize=(12, 8))

plt.subplot(3, 1, 1)
plt.plot(freq_axis[:N//2], 20 * np.log10(spec_lp[:N//2]))
plt.title('Lowpass Filtered Spectrum')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid()

plt.subplot(3, 1, 2)
plt.plot(freq_axis[:N//2], 20 * np.log10(spec_hp[:N//2]))
plt.title('Highpass Filtered Spectrum')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid()

plt.subplot(3, 1, 3)
plt.plot(freq_axis[:N//2], 20 * np.log10(spec_bp[:N//2]))
plt.title('Bandpass Filtered Spectrum')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid()

plt.tight_layout()
plt.show()
```



In [36]: `Audio(filtered_lp, rate = fs)`

Out[36]:

In [37]: `Audio(filtered_hp, rate = fs)`

Out[37]:

In [38]: `Audio(filtered_bp, rate = fs)`

Out[38]:

In [41]:

```
In [26]: import numpy as np
import scipy.signal as scs
from numpy.fft import fft
import matplotlib.pyplot as plt
from IPython.display import Audio
import scipy.io.wavfile as wave

fs_sputnik, audio_sputnik = wave.read('Sputnik.wav')
print(f'Sampling Rate Sputnik: {fs_sputnik} Hz')
print(f'Number of Samples Sputnik: {len(audio_sputnik)}')
print(f'Duration Sputnik: {len(audio_sputnik)/fs_sputnik:.2f} s')

audio_sputnik = audio_sputnik / np.max(np.abs(audio_sputnik))
```

```

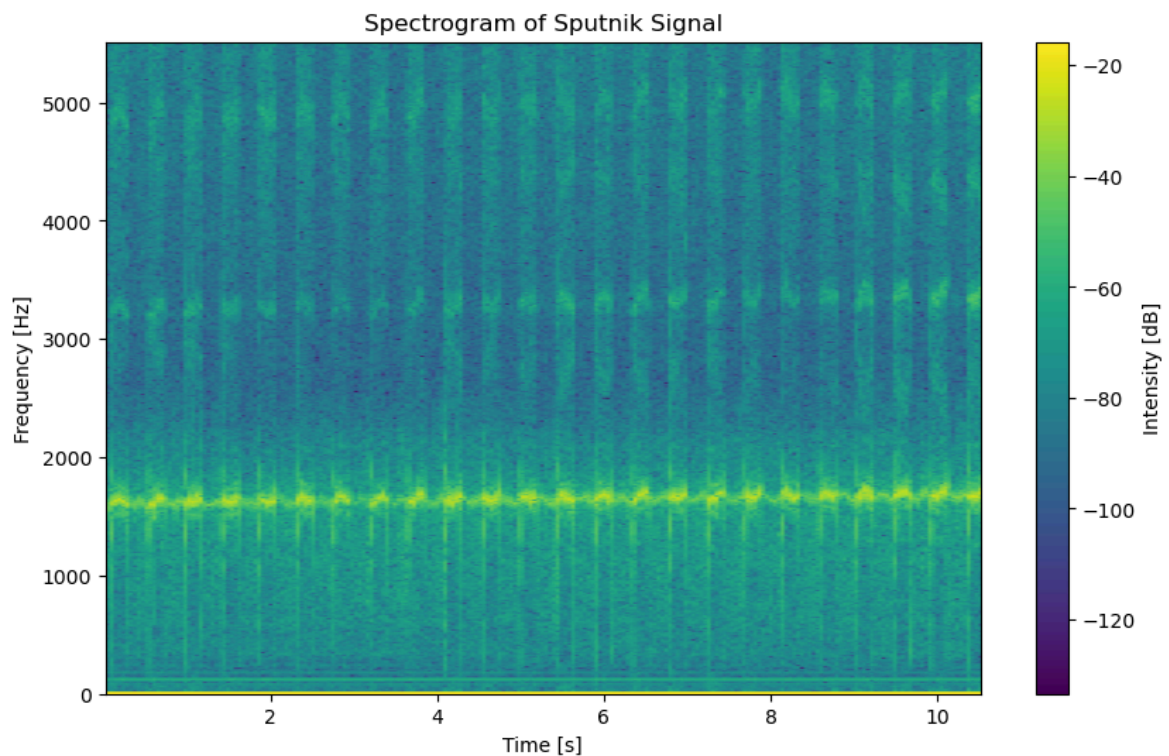
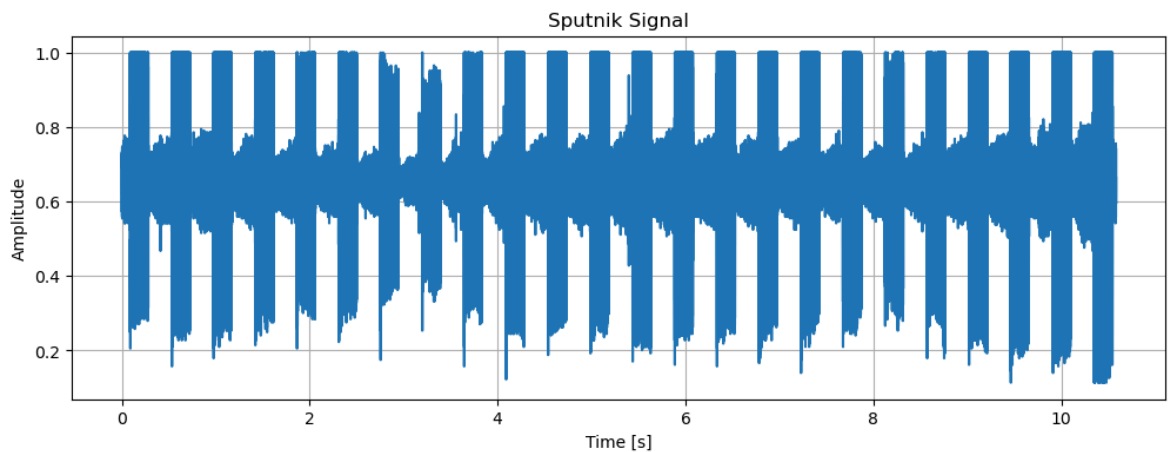
time_sputnik = np.arange(len(audio_sputnik)) / fs_sputnik

plt.figure(figsize=(12, 4))
plt.plot(time_sputnik, audio_sputnik)
plt.title('Sputnik Signal')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid()
plt.show()

plt.figure(figsize=(10, 6))
plt.specgram(audio_sputnik, NFFT=1024, Fs = fs_sputnik, noverlap=512)
plt.title('Spectrogram of Sputnik Signal')
plt.xlabel('Time [s]')
plt.ylabel('Frequency [Hz]')
plt.colorbar(label='Intensity [dB]')
plt.show()

```

Sampling Rate Sputnik: 11025 Hz
 Number of Samples Sputnik: 116672
 Duration Sputnik: 10.58 s



```

In [25]: spectrum_sputnik = np.abs(fft(audio_sputnik))
         freq_sputnik = np.arange(len(audio_sputnik)) * fs_sputnik / len(audio_sputnik)

```



```

peak_idx = np.argmax(spectrum_sputnik[:len(audio_sputnik)//2])
interference_freq = freq_sputnik[peak_idx]
print(f'Interference frequency: {interference_freq:.1f} Hz')

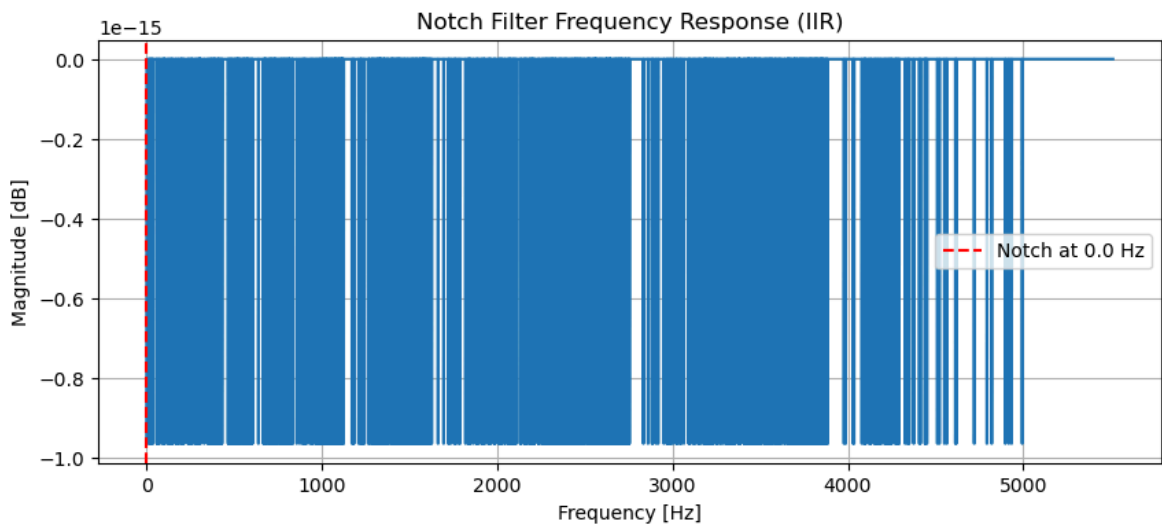
Q = 2000.0
b_notch, a_notch = scs.iirnotch(interference_freq, Q, fs_sputnik)

w_notch, h_notch = scs.freqz(b_notch, a_notch, worN=8000)
freq_notch = w_notch / np.pi * (fs_sputnik/2)

plt.figure(figsize=(10, 4))
plt.plot(freq_notch, 20 * np.log10(np.abs(h_notch)))
plt.title('Notch Filter Frequency Response (IIR)')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid()
plt.axvline(interference_freq, color='r', linestyle='--', label=f'Notch at {inte
plt.legend()
plt.show()

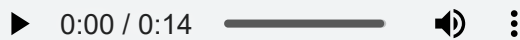
```

Interference frequency: 0.0 Hz



In [28]: `Audio(audio_sputnik, rate = fs)`

Out[28]:



In [29]: `filtered_sputnik = scs.lfilter(b_notch, a_notch, audio_sputnik)`

```

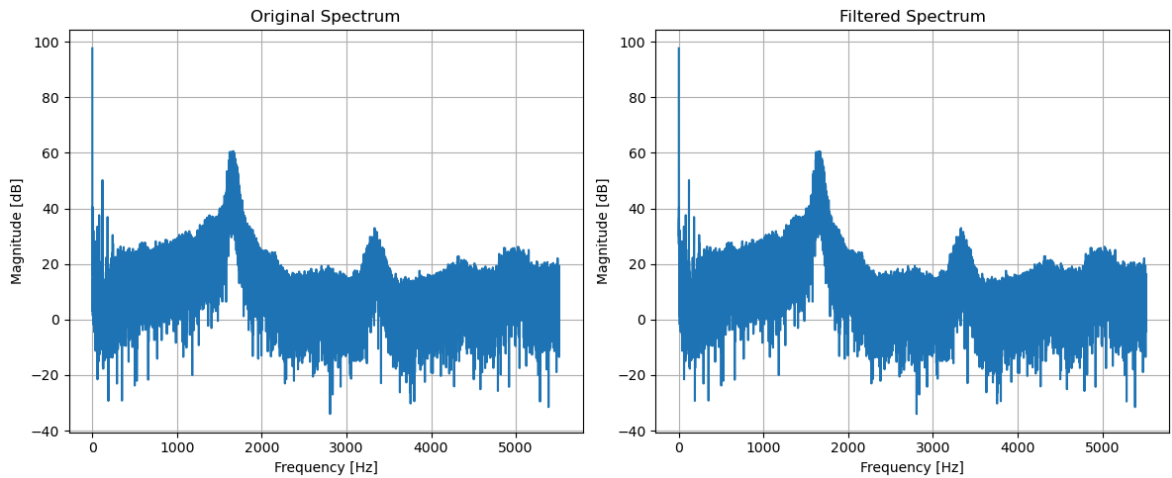
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(freq_sputnik[:len(audio_sputnik)//2], 20 * np.log10(spectrum_sputnik[:1
plt.title('Original Spectrum')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid()

plt.subplot(1, 2, 2)
spectrum_filtered = np.abs(fft(filtered_sputnik))
plt.plot(freq_sputnik[:len(audio_sputnik)//2], 20 * np.log10(spectrum_filtered[:1
plt.title('Filtered Spectrum')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')

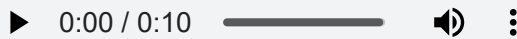
```

```
plt.grid()
plt.tight_layout()
plt.show()

Audio(filtered_sputnik, rate=fs_sputnik)
```



Out[29]:



In [33]:

```
fs_zara, audio_zara = wave.read('zarathustra.wav')
print(f'Sampling Rate Zarathustra: {fs_zara} Hz')
print(f'Number of Samples Zarathustra: {len(audio_zara)}')
print(f'Duration Zarathustra: {len(audio_zara)/fs_zara:.2f} s')

audio_zara_short = audio_zara[:10*fs_zara, 0] if audio_zara.ndim > 1 else audio_zara
audio_zara_short = audio_zara_short / np.max(np.abs(audio_zara_short))
time_zara = np.arange(len(audio_zara_short)) / fs_zara

spectrum_zara = np.abs(fft(audio_zara_short))
freq_zara = np.arange(len(audio_zara_short)) * fs_zara / len(audio_zara_short)

peak_idx_zara = np.argmax(spectrum_zara[:len(audio_zara_short)//2])
interference_freq_zara = freq_zara[peak_idx_zara]
print(f'Interference frequency in zarathustra: {interference_freq_zara:.1f} Hz')

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(time_zara, audio_zara_short)
plt.title('Zarathustra Signal (first 10 s)')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')

plt.subplot(1, 2, 2)
plt.plot(freq_zara[:len(audio_zara_short)//2], 20 * np.log10(spectrum_zara[:len(audio_zara_short)//2]))
plt.title('Spectrum')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.axvline(interference_freq_zara, color='r', linestyle='--', label=f'{interference_freq_zara:.1f} Hz')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

```

Q_zara = 2000.0
b_notch_zara, a_notch_zara = scs.iirnotch(interference_freq_zara, Q_zara, fs_zara)

filtered_zara = scs.lfilter(b_notch_zara, a_notch_zara, audio_zara_short)

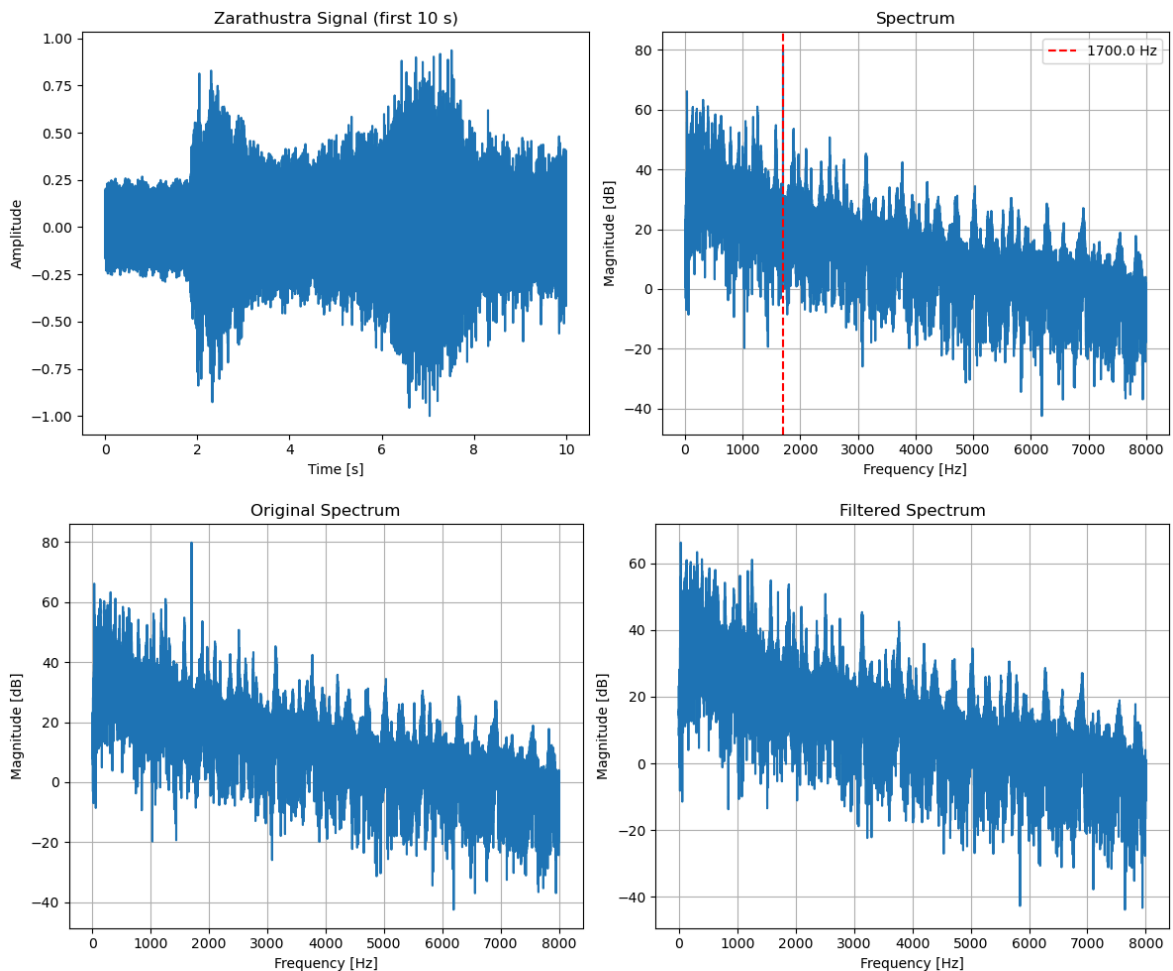
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(freq_zara[:len(audio_zara_short)//2], 20 * np.log10(spectrum_zara[:len(audio_zara_short)//2]))
plt.title('Original Spectrum')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid()

plt.subplot(1, 2, 2)
spectrum_filtered_zara = np.abs(fft(filtered_zara))
plt.plot(freq_zara[:len(audio_zara_short)//2], 20 * np.log10(spectrum_filtered_zara[:len(audio_zara_short)//2]))
plt.title('Filtered Spectrum')
plt.xlabel('Frequency [Hz]')
plt.ylabel('Magnitude [dB]')
plt.grid()
plt.tight_layout()
plt.show()

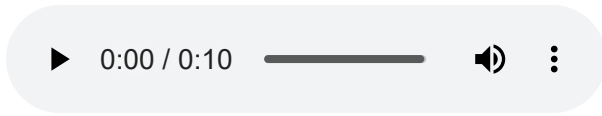
Audio(filtered_zara, rate=fs_zara)

```

Sampling Rate Zarathustra: 16000 Hz
 Number of Samples Zarathustra: 478820
 Duration Zarathustra: 29.93 s
 Interference frequency in zarathustra: 1700.0 Hz



Out[33]:



In []: